Agentic Sales Orchestrator - Pending Commercial Action Programmatic Extension

Customer-ready runbook with true line-by-line script explanation

Agentic Sales Orchestrator - Pending Commercial Action Programmatic Extension Runbook

Customer-ready implementation guide with line-by-line script explanation

Item	Value
Environment	Phoenicarix-CI
Dataverse URL	https://phoenicarix-ci.crm4.dynamics.com
Solution	ASO.Core
Solution unique name	ASOCore
Custom table	Pending Commercial Action
Table logical name	aso_pendingcommercialaction
Script purpose	Create the table, create business columns, attempt the Opportunity lookup relationship, publish customizations
Version	v1.0
Date	2026-05-24

Executive summary

We created the Pending Commercial Action custom Dataverse table programmatically in the Phoenicarix-CI environment and inside the ASO.Core solution. This table stages sensitive commercial actions before they are submitted to SAP or treated as commercially final. In plain language: it is a controlled waiting room where an order-related or pricing-related action can be prepared, reviewed, approved, submitted, and audited.

The first script successfully created the table and the main fields. It also attempted to create the missing Opportunity lookup relationship. That lookup attempt failed because the script used language code 1033, while the organization rejected that language for the relationship label operation. The fix was handled with a second focused script documented separately.

Business objective

The business objective was to protect the commercial/SAP boundary. Agentic Sales Orchestrator should not let AI, automation, or a seller workflow submit commercial actions directly to SAP without a controlled staging and approval pattern. The table therefore gives the project a structured place to store proposed commercial actions such as CreateOrder, UpdateCommercialReference, and ReviewPricingContext.

Technical objective

The technical objective was to create the custom Dataverse table aso_pendingcommercialaction and its columns in a repeatable, solution-aware way using a .NET 8 console script and the Dataverse SDK. The script targeted ASOCore so the created components belong to the correct unmanaged solution layer.

Scope

In scope:

- Create custom table `aso_pendingcommercialaction`.
- Create primary name column `aso_name`.
- Create table as user-owned.
- Create Action Type, Payload, Status, SAP Document ID, Error Message, Approval ID, Idempotency Key, and Submitted On columns.
- Attempt to create Opportunity lookup `aso_opportunityid`.
- Publish Pending Commercial Action and Opportunity metadata.
- Validate results in Power Apps.

Out of scope:

- No SAP API call.
- No Power Automate approval flow.
- No Customer Insights journey.
- No Foundry orchestration call.
- No form customization beyond validating columns and relationship.
- No real commercial submission.

Architecture/context

The table sits between Dynamics 365 Sales/Opportunity logic and the future governed SAP integration layer. It is not an integration itself. It is a staging table that future deterministic automation can use before calling APIM/Azure Functions/SAP.

```
Opportunity
  -> Pending Commercial Action
      -> Approval / review
          -> Governed SAP submit through APIM + Azure Functions
              -> SAP document/reference writeback
```

Why we used a programmatic approach instead of only the UI

Manual UI creation is possible, but this table includes multiple fields, choice values, metadata descriptions, solution-layering requirements, and a relationship to Opportunity. A script is faster, repeatable, auditable, and safer after validation. It also creates a reusable implementation pattern for the remaining custom ASO operational tables.

Environment and solution context

Area	Value
Environment	Phoenicarix-CI
URL	https://phoenicarix-ci.crm4.dynamics.com
Solution display name	ASO.Core
Solution unique name	ASOCore
Publisher prefix	aso
Local script folder	~/aso-dataverse-tools/pending-commercial-action

Prerequisites

- .NET 8 installed on the Mac.

- Dataverse SDK package `Microsoft.PowerPlatform.Dataverse.Client` added to the project.
- Maker/admin account can authenticate to Phoenicarix-CI.
- User has privileges to create tables, columns, and relationships. Usually System Administrator or System Customizer is required in the trial/MVP environment.
- `ASO.Core` exists and has unique name `ASOCore`.
- Managed and unmanaged backups of ASO.Core exist before schema changes.

Why managed and unmanaged backups matter

Before schema changes, we exported both unmanaged and managed ASO.Core packages. The unmanaged backup is useful for maker/developer recovery and inspection. The managed backup is useful to prove an exportable delivery package exists and to test downstream deployment behavior. Schema changes are not always simple to reverse, especially after dependencies, data, or relationships exist; backups give the team a recovery checkpoint.

Recommended backup names after this step:

```
ASO.Core_Phase2_CI_PendingCommercialActionCreated_unmanaged_v1_0_20260524.zip
ASO.Core_Phase2_CI_PendingCommercialActionCreated_managed_v1_0_20260524.zip
```

Field inventory created

Display name	Schema/logical name	Type	Values/size	Business purpose
Opportunity	aso_opportunityid	Lookup	Created by fix script	Links each pending commercial action to the Opportunity that originated or owns it.
Action Type	aso_actiontype	Local choice	CreateOrder; UpdateCommercialReference; ReviewPricingContext	Classifies the commercial action before approval and potential SAP submission.
Payload	aso_payload	Multiline text	4000	Stores the staged commercial payload; useful for approvals and audit.
Status	aso_status	Local choice	Draft; AwaitingApproval; Approved; Submitted; Failed; Cancelled	Tracks the approval/submission lifecycle of the action.
SAP Document ID	aso_sapdocumentid	Text	100	Stores the SAP document/reference returned after governed submission.
Error Message	aso_errormessage	Multiline text	4000	Captures failure details from approval/SAP submission processing.
Approval ID	aso_approvalid	Text	100	Stores the approval process identifier.
Idempotency Key	aso_idempotencykey	Text	200	Prevents duplicate submission of the same commercial action.
Submitted On	aso_submittedon	Date and time	User local	Timestamp of submission.

What happened during execution

The script created the table and the main fields successfully. During the Opportunity lookup creation, the terminal showed this error:

```
FAILED LOOKUP: aso_opportunityid
The language code 1033 is not a valid language for this organization
```

This means the relationship/lookup did not get created in the first script run. The table and its non-lookup fields were still created. We did not delete anything. Instead, we used a focused fix script to create only the missing Opportunity relationship with language code 1031.

Validation checklist

Check	Expected result
Table exists	ASO.Core -> Tables -> Pending Commercial Action is visible
Columns exist	Action Type, Payload, Status, SAP Document ID, Error Message, Approval ID, Idempotency Key, Submitted On
Relationship after fix	Relationship aso_opportunity_aso_pendingcommercialaction exists
Lookup after fix	Column aso_opportunityid / Opportunity exists
Publishing	Table and relationship are visible after refresh
Solution context	Components are inside ASO.Core

Troubleshooting guide

Symptom	Likely cause	Recommended action
Connection failed	Wrong URL, expired login, missing permission	Authenticate again and verify Phoenicarix-CI URL
Table already exists	Script was rerun	Safe; script skips existing table
Column already exists	Partial or repeated execution	Safe; script skips existing columns
Lookup fails with language code error	Label language not enabled for org	Use fix script with LanguageCode = 1031
Fields not in ASO.Core	Wrong SolutionUniqueName	Verify unique name with pac solution list
Build error/no entry point	Program.cs not fully pasted	Replace Program.cs with full script and save

Rollback/recovery approach

- If only the relationship failed: do not delete the table. Run the focused relationship fix script.
- If a non-production trial mistake is discovered immediately: delete the custom table only if there are no dependencies and no needed data.
- If dependencies already exist: remove dependent components first, then remove the table/columns.
- Use the latest unmanaged/managed solution backup as a checkpoint for comparison and recovery planning.
- In a real production ALM pipeline, rollback should be done through managed solution versioning and environment restore policies, not ad-hoc manual deletion.

What each audience should understand

Non-technical person

We created a safe waiting room for sensitive commercial actions. It does not send anything to SAP. It only stores the action so a person or process can review it first.

Product Owner

This table enables approval-gated commercial orchestration. It supports future SAP-safe order/reference handling without making AI or automation directly responsible for commercial submission.

Developer / technical consultant

This is Dataverse metadata automation. The script creates a user-owned custom table, string/memo/date/choice columns, and attempted a one-to-many Opportunity relationship through the SDK. The initial relationship operation failed because of the organization label language. A second script created the lookup with language code 1031.

Part 2 - Full script documentation with line-by-line explanation

Line	Code	Explanation in simple language	Technical explanation	Why it matters	Common mistake / warning
1	using Microsoft.Crm.Sdk.Messages;	Imports the Microsoft.Crm.Sdk.Messages namespace.	Allows the script to reference Dataverse SDK classes without fully qualified names.	Required for SDK request classes, metadata types, labels, and the ServiceClient connection.	Missing namespace causes compiler errors for the related SDK types.
2	using Microsoft.PowerPlatform.Dataverse.Client;	Imports the Microsoft.PowerPlatform.Dataverse.Client namespace.	Allows the script to reference Dataverse SDK classes without fully qualified names.	Required for SDK request classes, metadata types, labels, and the ServiceClient connection.	Missing namespace causes compiler errors for the related SDK types.
3	using Microsoft.Xrm.Sdk;	Imports the Microsoft.Xrm.Sdk namespace.	Allows the script to reference Dataverse SDK classes without fully qualified names.	Required for SDK request classes, metadata types, labels, and the ServiceClient connection.	Missing namespace causes compiler errors for the related SDK types.
4	using Microsoft.Xrm.Sdk.Messages;	Imports the Microsoft.Xrm.Sdk.Messages namespace.	Allows the script to reference Dataverse SDK classes without fully qualified names.	Required for SDK request classes, metadata types, labels, and the ServiceClient connection.	Missing namespace causes compiler errors for the related SDK types.
5	using Microsoft.Xrm.Sdk.Metadata;	Imports the Microsoft.Xrm.Sdk.Metadata namespace.	Allows the script to reference Dataverse SDK classes without fully qualified names.	Required for SDK request classes, metadata types, labels, and the ServiceClient connection.	Missing namespace causes compiler errors for the related SDK types.
6	(blank)	Blank readability line.	No runtime behavior; separates logical code blocks.	Improves maintainability and visual scanning.	None.
7	const string DataverseUrl = "https://phoenicarix-ci.crm4.dynamics.com";	Defines the target Dataverse environment URL.	ServiceClient uses this URL to connect to Phoenicarix-CI.	Prevents accidental schema creation in the wrong tenant/environment.	Wrong URL creates components in the wrong environment or fails authentication.
8	const string SolutionUniqueName =	Defines the technical solution unique name:	CreateEntityRequest,	Keeps created components inside the	Using display name instead of unique name

	"ASOCore";	ASOCore.	CreateAttributeRequest, and CreateOneToManyRequest use this to associate metadata with ASO.Core.	project solution for ALM/export.	can leave components outside the intended solution.
9	const string EntityLogicalName = "aso_pendingcommercialaction";	Defines the target table logical name.	The script uses aso_pendingcommercialaction as the Dataverse table logical name.	Every table, column, relationship, and publish request depends on this value.	Wrong logical name targets the wrong table or causes metadata request failures.
10	const int LanguageCode = 1033;	Sets English language code 1033 for labels.	Label(...) uses this language code for display names and descriptions.	Worked for table/column labels, but failed for the relationship label in this organization.	This was the source of the lookup error: '1033 is not a valid language for this organization'.
11	(blank)	Blank readability line.	No runtime behavior; separates logical code blocks.	Improves maintainability and visual scanning.	None.
12	var connectionString =	Starts the Dataverse connection string definition.	The following lines define OAuth authentication, URL, login behavior, client ID, and redirect URI.	The script needs an authenticated ServiceClient to execute metadata operations.	Incomplete connection strings prevent login or service initialization.
13	$@"AuthType=OAuth;	Specifies OAuth authentication.	OAuth is the interactive sign-in method for the local Dataverse SDK connection.	Allows a maker/admin to sign in with their Power Platform account.	Wrong auth type may require unavailable secrets or fail on Mac/local use.
14	Url={DataverseUrl};	Injects the configured Dataverse URL into the connection string.	Uses the DataverseUrl constant rather than hardcoding the URL multiple times.	Makes environment targeting explicit and maintainable.	If the URL is wrong, all metadata operations target the wrong environment.
15	LoginPrompt=Auto;	Allows interactive login when needed.	ServiceClient can open a login flow if no valid token is cached.	Practical for a Mac-based maker/developer working locally.	Without a login prompt, the script may fail silently when no token is available.
16	ClientId=51f81489-12ee-4a9e-aaae-a2591f45987d;	Sets the Microsoft public client ID used by Dataverse tooling scenarios.	ServiceClient uses it for the interactive OAuth flow.	Avoids creating a custom app registration for this MVP metadata script.	Changing it to an invalid client ID breaks authentication.
17	RedirectUri=http://localhost";	Defines the local redirect URI for interactive OAuth.	After sign-in, authentication returns to localhost so the SDK can continue.	Required by the OAuth flow used in this local script.	Incorrect redirect URI can block authentication completion.
18	(blank)	Blank readability line.	No runtime behavior; separates logical code blocks.	Improves maintainability and visual scanning.	None.
19	using var service = new ServiceClient(connectionString);	Imports the var service = new ServiceClient(connectionString) namespace.	Allows the script to reference Dataverse SDK classes without fully qualified names.	Required for SDK request classes, metadata types, labels, and the ServiceClient connection.	Missing namespace causes compiler errors for the related SDK types.
20	(blank)	Blank readability line.	No runtime behavior; separates logical code blocks.	Improves maintainability and visual scanning.	None.
21	if (!service.IsReady)	Checks whether connection initialization succeeded.	Prevents metadata operations when authentication or connectivity failed.	Protects the environment from partial/undefined behavior.	Skipping this check can lead to confusing failures later in the script.
22	{	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
23	Console.ForegroundColor = ConsoleColor.Red;	Changes terminal output to red.	Used for failure messages so operators can identify errors immediately.	Improves operational readability.	If colors are not reset, later messages can remain incorrectly colored.
24	Console.WriteLine("Connection failed:");	Writes a status message to the terminal.	Provides an execution log for the operator.	Makes the run auditable and easier to troubleshoot.	If messages are vague, failures become harder to interpret.
25	Console.WriteLine(service.LastError);	Prints the Dataverse connection error.	ServiceClient.LastError provides diagnostic detail when connection fails.	Needed for troubleshooting URL, login, permission, or token issues.	Without it, the operator sees only a generic failure.
26	Console.ResetColor();	Executes part of the script's metadata creation logic.	This line participates in defining, checking, creating, logging, or publishing Dataverse metadata.	It contributes to the controlled creation of the Pending Commercial Action table or lookup fix.	If changed, validate compile output and Power Apps metadata carefully.
27	return;	Stops script execution.	Used after a connection failure or after skipping lookup creation.	Prevents unsafe continuation or exits a helper method when work is unnecessary.	Removing it can cause the script to run into invalid state.
28	}	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
29	(blank)	Blank readability line.	No runtime behavior; separates logical code blocks.	Improves maintainability and visual scanning.	None.
30	Console.WriteLine($"Connected to: {DataverseUrl}");	Writes a status message to the terminal.	Provides an execution log for the operator.	Makes the run auditable and easier to troubleshoot.	If messages are vague, failures become harder to interpret.
31	Console.WriteLine($"Target solution: {SolutionUniqueName}");	Writes a status message to the terminal.	Provides an execution log for the operator.	Makes the run auditable and easier to troubleshoot.	If messages are vague, failures become harder to interpret.
32	Console.WriteLine($"Target table: {EntityLogicalName}");	Writes a status message to the terminal.	Provides an execution log for the operator.	Makes the run auditable and easier to troubleshoot.	If messages are vague, failures become harder to interpret.
33	Console.WriteLine();	Writes a status message to the terminal.	Provides an execution log for the operator.	Makes the run auditable and easier to troubleshoot.	If messages are vague, failures become harder to interpret.
34	(blank)	Blank readability line.	No runtime behavior; separates logical code blocks.	Improves maintainability and visual scanning.	None.
35	if (!EntityExists(service, EntityLogicalName))	Checks whether the Pending Commercial Action table already exists.	Uses a RetrieveEntityRequest helper; if the table is missing, the script creates it.	Makes the script safer to run in a partially completed environment.	Without this, rerunning the script could fail on duplicate table creation.
36	{	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
37	Console.WriteLine("Creating Pending Commercial Action table...");	Writes a status message to the terminal.	Provides an execution log for the operator.	Makes the run auditable and easier to troubleshoot.	If messages are vague, failures become harder to interpret.
38	(blank)	Blank readability line.	No runtime behavior; separates logical code blocks.	Improves maintainability and visual scanning.	None.
39	var createEntityRequest = new CreateEntityRequest	Creates a Dataverse table creation request.	The request carries table metadata, primary attribute metadata, and the target solution unique name.	This is the SDK mechanism that creates the custom table.	Incorrect request configuration creates the wrong table shape or fails validation.
40	{	Structural C# syntax.	Groups code into blocks or closes object	Required for valid C# compilation and scope	Missing braces or semicolons cause build

			initializers/statements.	control.	errors.
41	Entity = new EntityMetadata	Starts table metadata definition.	EntityMetadata describes the custom table display names, schema name, description, and ownership model.	Defines what Dataverse table is created.	Wrong metadata can create an incorrectly named or incorrectly owned table.
42	{	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
43	SchemaName = "aso_pendingcommercialaction",	Sets the table schema name.	Dataverse uses this as the technical name for the custom table.	Ensures the table follows the ASO publisher prefix and Phase 2 model.	Changing it creates a different table than the guide expects.
44	DisplayName = Label("Pending Commercial Action"),	Sets the singular display name.	The Label helper creates a localized Dataverse label.	Users see this name in the maker portal and model-driven apps.	Unsupported language code can cause label creation failure.
45	DisplayCollectionName = Label("Pending Commercial Actions"),	Sets the plural display name.	Used by Dataverse for table lists and navigation.	Improves readability in the UI.	A poor label confuses makers and admins.
46	Description = Label("Approval-gated commercial action staging table before SAP submit."),	Adds a metadata description.	Dataverse stores this explanation on the table or column metadata.	Helps future IT teams understand intent.	Descriptions are easy to skip but important for maintainability.
47	OwnershipType = OwnershipTypes.UserOwned	Sets the table as user/team-owned.	Records have owners and can participate in security ownership behavior.	Appropriate because pending commercial actions may be assigned/owned by users or teams.	Wrong ownership changes security and assignment behavior.
48	},	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
49	PrimaryAttribute = new StringAttributeMetadata	Starts the primary name column definition.	Every custom Dataverse table needs a primary name attribute.	Provides the human-readable main column for the table.	Missing or invalid primary attribute prevents table creation.
50	{	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
51	SchemaName = "aso_name",	Defines the primary name column schema.	aso_name is the main name field for the custom table.	Keeps the table aligned with ASO prefixing.	Wrong primary schema creates inconsistent metadata.
52	DisplayName = Label("Name"),	Executes part of the script's metadata creation logic.	This line participates in defining, checking, creating, logging, or publishing Dataverse metadata.	It contributes to the controlled creation of the Pending Commercial Action table or lookup fix.	If changed, validate compile output and Power Apps metadata carefully.
53	Description = Label("Primary name for the Pending Commercial Action record."),	Adds a metadata description.	Dataverse stores this explanation on the table or column metadata.	Helps future IT teams understand intent.	Descriptions are easy to skip but important for maintainability.
54	RequiredLevel = new AttributeRequiredLevelManagedProperty(AttributeRequiredLevel.ApplicationRequired),	Makes the primary name application-required.	Dataverse requires a primary name for reliable row identification.	Ensures records have a readable identifier.	Overly strict requirements on other fields could hurt automation, but primary name is appropriate.
55	MaxLength = 200	Sets the maximum text length.	Dataverse validates text length at metadata and runtime levels.	Prevents uncontrolled field sizes and follows the data model.	Too small truncates useful data; too large may reduce data quality.
56	},	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
57	SolutionUniqueName = SolutionUniqueName	Associates the created component with ASO.Core.	The SDK passes the solution unique name to the create request.	Critical for export, ALM, and keeping changes out of accidental default-only context.	Wrong solution unique name means components may not appear in ASO.Core.
58	};	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
59	(blank)	Blank readability line.	No runtime behavior; separates logical code blocks.	Improves maintainability and visual scanning.	None.
60	service.Execute(createEntityRequest);	Executes the custom table creation request.	Sends the metadata request to Dataverse.	This is the line that actually creates the table.	If permissions are missing, this line fails.
61	(blank)	Blank readability line.	No runtime behavior; separates logical code blocks.	Improves maintainability and visual scanning.	None.
62	Console.ForegroundColor = ConsoleColor.Green;	Changes terminal output to green.	Used for successful creation/publishing messages.	Gives visual confirmation of completed metadata actions.	Purely cosmetic, but useful during long script runs.
63	Console.WriteLine("CREATED TABLE: aso_pendingcommercialaction");	Writes a status message to the terminal.	Provides an execution log for the operator.	Makes the run auditable and easier to troubleshoot.	If messages are vague, failures become harder to interpret.
64	Console.ResetColor();	Executes part of the script's metadata creation logic.	This line participates in defining, checking, creating, logging, or publishing Dataverse metadata.	It contributes to the controlled creation of the Pending Commercial Action table or lookup fix.	If changed, validate compile output and Power Apps metadata carefully.
65	}	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
66	else	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
67	{	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
68	Console.ForegroundColor = ConsoleColor.Yellow;	Changes terminal output to yellow.	Used for skip messages when a component already exists.	Helps distinguish safe idempotent skips from failures.	Purely cosmetic, but useful during reruns.
69	Console.WriteLine("SKIP existing table: aso_pendingcommercialaction");	Writes a status message to the terminal.	Provides an execution log for the operator.	Makes the run auditable and easier to troubleshoot.	If messages are vague, failures become harder to interpret.
70	Console.ResetColor();	Executes part of the script's metadata creation logic.	This line participates in defining, checking, creating, logging, or publishing Dataverse metadata.	It contributes to the controlled creation of the Pending Commercial Action table or lookup fix.	If changed, validate compile output and Power Apps metadata carefully.
71	}	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.

72	(blank)	Blank readability line.	No runtime behavior; separates logical code blocks.	Improves maintainability and visual scanning.	None.
73	CreateOpportunityLookupIfMissing(service);	Calls the helper that tries to create the Opportunity lookup.	The helper checks for aso_opportunityid and creates a many-to-one relationship if missing.	Links pending commercial actions to opportunities.	This failed in the initial script due to unsupported language code 1033.
74	(blank)	Blank readability line.	No runtime behavior; separates logical code blocks.	Improves maintainability and visual scanning.	None.
75	var existingAttributes = GetExistingAttributeLogicalNames(service, EntityLogicalName);	Reads existing columns on the target table.	The helper retrieves Attribute metadata and returns logical names.	Enables idempotent reruns that skip already-created columns.	Without this, reruns would fail on duplicate column names.
76	(blank)	Blank readability line.	No runtime behavior; separates logical code blocks.	Improves maintainability and visual scanning.	None.
77	var fields = new List<AttributeMetadata>	Starts an in-memory list of columns to create.	Each item is an AttributeMetadata object produced by helper functions.	Keeps the field inventory declarative and reviewable.	Bad entries here create wrong schema.
78	{	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
79	LocalChoice("aso_actiontype", "Action Type",	Defines a local choice column.	Creates a PicklistAttributeMetadata object with values embedded in this one column.	Used when the option set is specific to Pending Commercial Action.	If values are wrong, automation status/action logic becomes inconsistent.
80	new[] { "CreateOrder", "UpdateCommercialReference", "ReviewPricingContext" },	Executes part of the script's metadata creation logic.	This line participates in defining, checking, creating, logging, or publishing Dataverse metadata.	It contributes to the controlled creation of the Pending Commercial Action table or lookup fix.	If changed, validate compile output and Power Apps metadata carefully.
81	"Commercial action type staged for approval before SAP submission."),	Executes part of the script's metadata creation logic.	This line participates in defining, checking, creating, logging, or publishing Dataverse metadata.	It contributes to the controlled creation of the Pending Commercial Action table or lookup fix.	If changed, validate compile output and Power Apps metadata carefully.
82	(blank)	Blank readability line.	No runtime behavior; separates logical code blocks.	Improves maintainability and visual scanning.	None.
83	Memo("aso_payload", "Payload", "Commercial action payload staged before approval and SAP submission."),	Defines a multiline text column.	Creates MemoAttributeMetadata with 4000-character capacity.	Used for payloads and error text that may be longer than normal text.	Using short text here could truncate important diagnostics or payload data.
84	(blank)	Blank readability line.	No runtime behavior; separates logical code blocks.	Improves maintainability and visual scanning.	None.
85	LocalChoice("aso_status", "Status",	Defines a local choice column.	Creates a PicklistAttributeMetadata object with values embedded in this one column.	Used when the option set is specific to Pending Commercial Action.	If values are wrong, automation status/action logic becomes inconsistent.
86	new[] { "Draft", "AwaitingApproval", "Approved", "Submitted", "Failed", "Cancelled" },	Executes part of the script's metadata creation logic.	This line participates in defining, checking, creating, logging, or publishing Dataverse metadata.	It contributes to the controlled creation of the Pending Commercial Action table or lookup fix.	If changed, validate compile output and Power Apps metadata carefully.
87	"Approval and submission status of the pending commercial action."),	Executes part of the script's metadata creation logic.	This line participates in defining, checking, creating, logging, or publishing Dataverse metadata.	It contributes to the controlled creation of the Pending Commercial Action table or lookup fix.	If changed, validate compile output and Power Apps metadata carefully.
88	(blank)	Blank readability line.	No runtime behavior; separates logical code blocks.	Improves maintainability and visual scanning.	None.
89	Text("aso_sapdocumentid", "SAP Document ID", 100, "SAP document identifier returned after governed SAP submit."),	Defines a single-line text column.	Creates StringAttributeMetadata with a defined MaxLength.	Used for IDs, keys, and references.	Wrong length can truncate identifiers or allow poor-quality oversized values.
90	Memo("aso_errormessage", "Error Message", "Error details if approval or SAP submission fails."),	Defines a multiline text column.	Creates MemoAttributeMetadata with 4000-character capacity.	Used for payloads and error text that may be longer than normal text.	Using short text here could truncate important diagnostics or payload data.
91	Text("aso_approvalid", "Approval ID", 100, "Power Automate or approval process identifier."),	Defines a single-line text column.	Creates StringAttributeMetadata with a defined MaxLength.	Used for IDs, keys, and references.	Wrong length can truncate identifiers or allow poor-quality oversized values.
92	Text("aso_idempotencykey", "Idempotency Key", 200, "Key used to prevent duplicate commercial submission."),	Defines a single-line text column.	Creates StringAttributeMetadata with a defined MaxLength.	Used for IDs, keys, and references.	Wrong length can truncate identifiers or allow poor-quality oversized values.
93	DateTimeField("aso_submittedon", "Submitted On", "Timestamp when the commercial action was submitted.")	Defines a date/time column.	Creates DateTimeAttributeMetadata with DateAndTime format and UserLocal behavior.	Used for Submitted On timestamps.	Wrong behavior can confuse users across time zones.
94	};	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
95	(blank)	Blank readability line.	No runtime behavior; separates logical code blocks.	Improves maintainability and visual scanning.	None.
96	foreach (var field in fields)	Starts a loop through all planned columns.	Each field definition is processed one by one.	Avoids repetitive manual create calls and standardizes behavior.	Loop logic must handle existing fields and errors safely.
97	{	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
98	var logicalName = field.SchemaName!.ToLowerInvariant();	Reads and normalizes the planned field schema name.	Dataverse logical names are lowercase, so ToLowerInvariant supports reliable comparison.	Enables correct duplicate detection.	Incorrect comparison could attempt duplicate creation.
99	(blank)	Blank readability line.	No runtime behavior; separates logical code blocks.	Improves maintainability and visual scanning.	None.
100	if (existingAttributes.Contains(logicalName))	Checks whether a column already exists.	Uses the current table metadata returned earlier.	Makes the script rerunnable after partial success.	Without it, duplicate column errors stop progress.

101	{	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
102	Console.ForegroundColor = ConsoleColor.Yellow;	Changes terminal output to yellow.	Used for skip messages when a component already exists.	Helps distinguish safe idempotent skips from failures.	Purely cosmetic, but useful during reruns.
103	Console.WriteLine($"SKIP existing: {logicalName}");	Writes a status message to the terminal.	Provides an execution log for the operator.	Makes the run auditable and easier to troubleshoot.	If messages are vague, failures become harder to interpret.
104	Console.ResetColor();	Executes part of the script's metadata creation logic.	This line participates in defining, checking, creating, logging, or publishing Dataverse metadata.	It contributes to the controlled creation of the Pending Commercial Action table or lookup fix.	If changed, validate compile output and Power Apps metadata carefully.
105	continue;	Skips the rest of the current loop iteration.	After detecting an existing column, the script moves to the next field.	Supports safe reruns.	Removing it would still try to create an existing column.
106	}	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
107	(blank)	Blank readability line.	No runtime behavior; separates logical code blocks.	Improves maintainability and visual scanning.	None.
108	try	Starts a protected operation block.	Exceptions inside the block are caught and logged.	Prevents one failing field from hiding the exact cause.	Without try/catch, the script may crash with less context.
109	{	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
110	Console.WriteLine($"Creating: {logicalName} ...");	Writes a status message to the terminal.	Provides an execution log for the operator.	Makes the run auditable and easier to troubleshoot.	If messages are vague, failures become harder to interpret.
111	(blank)	Blank readability line.	No runtime behavior; separates logical code blocks.	Improves maintainability and visual scanning.	None.
112	var request = new CreateAttributeRequest	Creates a column creation request.	Dataverse metadata columns are created with CreateAttributeRequest.	This is how the script adds each planned column.	Wrong EntityName or Attribute creates the wrong metadata or fails.
113	{	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
114	EntityName = EntityLogicalName,	Targets the Pending Commercial Action table.	Uses the table logical name constant.	Ensures columns are created on aso_pendingcommercialaction.	Wrong target places fields on the wrong table.
115	Attribute = field,	Attaches the current field metadata to the request.	The current AttributeMetadata object contains schema, type, label, and settings.	This is the column being created.	If field is malformed, the request fails.
116	SolutionUniqueName = SolutionUniqueName	Associates the created component with ASO.Core.	The SDK passes the solution unique name to the create request.	Critical for export, ALM, and keeping changes out of accidental default-only context.	Wrong solution unique name means components may not appear in ASO.Core.
117	};	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
118	(blank)	Blank readability line.	No runtime behavior; separates logical code blocks.	Improves maintainability and visual scanning.	None.
119	service.Execute(request);	Executes a create request.	Sends CreateAttributeRequest or CreateOneToManyRequest to Dataverse.	Actually creates a column or relationship.	Failures here indicate permissions, invalid metadata, or language/choice issues.
120	(blank)	Blank readability line.	No runtime behavior; separates logical code blocks.	Improves maintainability and visual scanning.	None.
121	Console.ForegroundColor = ConsoleColor.Green;	Changes terminal output to green.	Used for successful creation/publishing messages.	Gives visual confirmation of completed metadata actions.	Purely cosmetic, but useful during long script runs.
122	Console.WriteLine($"CREATED: {logicalName}");	Writes a status message to the terminal.	Provides an execution log for the operator.	Makes the run auditable and easier to troubleshoot.	If messages are vague, failures become harder to interpret.
123	Console.ResetColor();	Executes part of the script's metadata creation logic.	This line participates in defining, checking, creating, logging, or publishing Dataverse metadata.	It contributes to the controlled creation of the Pending Commercial Action table or lookup fix.	If changed, validate compile output and Power Apps metadata carefully.
124	}	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
125	catch (Exception ex)	Starts exception handling.	Captures SDK/Dataverse errors for the current operation.	Keeps the operator informed about exact failures.	Ignoring exceptions makes recovery difficult.
126	{	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
127	Console.ForegroundColor = ConsoleColor.Red;	Changes terminal output to red.	Used for failure messages so operators can identify errors immediately.	Improves operational readability.	If colors are not reset, later messages can remain incorrectly colored.
128	Console.WriteLine($"FAILED: {logicalName}");	Writes a status message to the terminal.	Provides an execution log for the operator.	Makes the run auditable and easier to troubleshoot.	If messages are vague, failures become harder to interpret.
129	Console.WriteLine(ex.Message);	Writes a status message to the terminal.	Provides an execution log for the operator.	Makes the run auditable and easier to troubleshoot.	If messages are vague, failures become harder to interpret.
130	Console.ResetColor();	Executes part of the script's metadata creation logic.	This line participates in defining, checking, creating, logging, or publishing Dataverse metadata.	It contributes to the controlled creation of the Pending Commercial Action table or lookup fix.	If changed, validate compile output and Power Apps metadata carefully.
131	}	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
132	}	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
133	(blank)	Blank readability line.	No runtime behavior; separates logical code blocks.	Improves maintainability and visual scanning.	None.
134	Console.WriteLine();	Writes a status message to the terminal.	Provides an execution log for the operator.	Makes the run auditable and easier to troubleshoot.	If messages are vague, failures become harder to interpret.

135	Console.WriteLine("Publishing Pending Commercial Action table customizations...");	Writes a status message to the terminal.	Provides an execution log for the operator.	Makes the run auditable and easier to troubleshoot.	If messages are vague, failures become harder to interpret.
136	(blank)	Blank readability line.	No runtime behavior; separates logical code blocks.	Improves maintainability and visual scanning.	None.
137	service.Execute(new PublishXmlRequest	Creates a targeted publish request.	Publishes metadata changes for specific entities rather than everything.	Makes created columns/relationships visible and usable in the maker portal.	Without publish, changes may not appear immediately.
138	{	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
139	ParameterXml = "<importexportxml><entities><entity>aso_pendingcommercialaction</entity><entity>opportunity</entity></entities></importexportxml>"	Defines which tables to publish.	The XML lists aso_pendingcommercialaction and opportunity.	Both sides are relevant because a relationship touches both tables.	Wrong XML may publish too little or fail.
140	});	Executes part of the script's metadata creation logic.	This line participates in defining, checking, creating, logging, or publishing Dataverse metadata.	It contributes to the controlled creation of the Pending Commercial Action table or lookup fix.	If changed, validate compile output and Power Apps metadata carefully.
141	(blank)	Blank readability line.	No runtime behavior; separates logical code blocks.	Improves maintainability and visual scanning.	None.
142	Console.ForegroundColor = ConsoleColor.Green;	Changes terminal output to green.	Used for successful creation/publishing messages.	Gives visual confirmation of completed metadata actions.	Purely cosmetic, but useful during long script runs.
143	Console.WriteLine("Done. Validate in ASO.Core -> Tables -> Pending Commercial Action -> Columns.");	Writes a status message to the terminal.	Provides an execution log for the operator.	Makes the run auditable and easier to troubleshoot.	If messages are vague, failures become harder to interpret.
144	Console.ResetColor();	Executes part of the script's metadata creation logic.	This line participates in defining, checking, creating, logging, or publishing Dataverse metadata.	It contributes to the controlled creation of the Pending Commercial Action table or lookup fix.	If changed, validate compile output and Power Apps metadata carefully.
145	(blank)	Blank readability line.	No runtime behavior; separates logical code blocks.	Improves maintainability and visual scanning.	None.
146	void CreateOpportunityLookupIfMissing(ServiceClient service)	Defines a helper method for the Opportunity relationship.	It checks for aso_opportunityid and creates a many-to-one lookup if absent.	Keeps relationship logic separate from field creation logic.	If omitted, the table exists but cannot link records to opportunities.
147	{	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
148	var existingAttributes = GetExistingAttributeLogicalNames(service, EntityLogicalName);	Reads existing columns on the target table.	The helper retrieves Attribute metadata and returns logical names.	Enables idempotent reruns that skip already-created columns.	Without this, reruns would fail on duplicate column names.
149	(blank)	Blank readability line.	No runtime behavior; separates logical code blocks.	Improves maintainability and visual scanning.	None.
150	if (existingAttributes.Contains("aso_opportunityid"))	Checks whether a column already exists.	Uses the current table metadata returned earlier.	Makes the script rerunnable after partial success.	Without it, duplicate column errors stop progress.
151	{	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
152	Console.ForegroundColor = ConsoleColor.Yellow;	Changes terminal output to yellow.	Used for skip messages when a component already exists.	Helps distinguish safe idempotent skips from failures.	Purely cosmetic, but useful during reruns.
153	Console.WriteLine("SKIP existing lookup: aso_opportunityid");	Writes a status message to the terminal.	Provides an execution log for the operator.	Makes the run auditable and easier to troubleshoot.	If messages are vague, failures become harder to interpret.
154	Console.ResetColor();	Executes part of the script's metadata creation logic.	This line participates in defining, checking, creating, logging, or publishing Dataverse metadata.	It contributes to the controlled creation of the Pending Commercial Action table or lookup fix.	If changed, validate compile output and Power Apps metadata carefully.
155	return;	Stops script execution.	Used after a connection failure or after skipping lookup creation.	Prevents unsafe continuation or exits a helper method when work is unnecessary.	Removing it can cause the script to run into invalid state.
156	}	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
157	(blank)	Blank readability line.	No runtime behavior; separates logical code blocks.	Improves maintainability and visual scanning.	None.
158	try	Starts a protected operation block.	Exceptions inside the block are caught and logged.	Prevents one failing field from hiding the exact cause.	Without try/catch, the script may crash with less context.
159	{	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
160	Console.WriteLine("Creating Opportunity lookup relationship...");	Writes a status message to the terminal.	Provides an execution log for the operator.	Makes the run auditable and easier to troubleshoot.	If messages are vague, failures become harder to interpret.
161	(blank)	Blank readability line.	No runtime behavior; separates logical code blocks.	Improves maintainability and visual scanning.	None.
162	var relationship = new OneToManyRelationshipMetadata	Starts a one-to-many relationship definition.	From Opportunity to Pending Commercial Action: one Opportunity can have many pending actions.	This models the business relationship correctly.	Wrong relationship direction affects forms, lookups, and related records.
163	{	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
164	SchemaName = "aso_opportunity_aso_pendingcommercialaction",	Sets the relationship schema name.	Dataverse stores relationships with schema names.	Allows the relationship to be identified and transported in solutions.	Changing the name creates a different relationship.

165	ReferencedEntity = "opportunity",	Sets Opportunity as the parent/referenced table.	Opportunity is the table being looked up to.	Each pending action belongs to one opportunity.	Wrong referenced table breaks the business model.
166	ReferencingEntity = EntityLogicalName,	Sets Pending Commercial Action as the child/referencing table.	The lookup column lives on the Pending Commercial Action table.	This lets a pending action point to an Opportunity.	Wrong referencing entity places the lookup elsewhere.
167	AssociatedMenuConfiguration = new AssociatedMenuConfiguration	Defines how related records appear in navigation.	Controls the related menu label/group/order in model-driven apps.	Improves user navigation from Opportunity to pending actions.	Label language issues can fail this part of relationship creation.
168	{	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
169	Behavior = AssociatedMenuBehavior.UseLabel,	Executes part of the script's metadata creation logic.	This line participates in defining, checking, creating, logging, or publishing Dataverse metadata.	It contributes to the controlled creation of the Pending Commercial Action table or lookup fix.	If changed, validate compile output and Power Apps metadata carefully.
170	Group = AssociatedMenuGroup.Details,	Executes part of the script's metadata creation logic.	This line participates in defining, checking, creating, logging, or publishing Dataverse metadata.	It contributes to the controlled creation of the Pending Commercial Action table or lookup fix.	If changed, validate compile output and Power Apps metadata carefully.
171	Label = Label("Pending Commercial Actions"),	Sets the related menu label.	Uses the Label helper and current LanguageCode.	This was where unsupported 1033 contributed to the first failure.	Use an enabled organization language such as 1031 in this tenant.
172	Order = 10000	Executes part of the script's metadata creation logic.	This line participates in defining, checking, creating, logging, or publishing Dataverse metadata.	It contributes to the controlled creation of the Pending Commercial Action table or lookup fix.	If changed, validate compile output and Power Apps metadata carefully.
173	},	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
174	CascadeConfiguration = new CascadeConfiguration	Starts cascade behavior configuration.	Defines what happens to child records when the parent opportunity is assigned, deleted, merged, shared, etc.	Protects commercial audit records from accidental destructive cascades.	Aggressive cascade deletes could remove important approval/audit data.
175	{	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
176	Assign = CascadeType.NoCascade,	Executes part of the script's metadata creation logic.	This line participates in defining, checking, creating, logging, or publishing Dataverse metadata.	It contributes to the controlled creation of the Pending Commercial Action table or lookup fix.	If changed, validate compile output and Power Apps metadata carefully.
177	Delete = CascadeType.RemoveLink,	Configures delete behavior to remove the link.	If the parent is deleted, Dataverse removes the relationship link rather than cascading deletion.	Preserves pending action records where possible.	A cascade delete could erase audit data.
178	Merge = CascadeType.NoCascade,	Executes part of the script's metadata creation logic.	This line participates in defining, checking, creating, logging, or publishing Dataverse metadata.	It contributes to the controlled creation of the Pending Commercial Action table or lookup fix.	If changed, validate compile output and Power Apps metadata carefully.
179	Reparent = CascadeType.NoCascade,	Executes part of the script's metadata creation logic.	This line participates in defining, checking, creating, logging, or publishing Dataverse metadata.	It contributes to the controlled creation of the Pending Commercial Action table or lookup fix.	If changed, validate compile output and Power Apps metadata carefully.
180	Share = CascadeType.NoCascade,	Executes part of the script's metadata creation logic.	This line participates in defining, checking, creating, logging, or publishing Dataverse metadata.	It contributes to the controlled creation of the Pending Commercial Action table or lookup fix.	If changed, validate compile output and Power Apps metadata carefully.
181	Unshare = CascadeType.NoCascade	Executes part of the script's metadata creation logic.	This line participates in defining, checking, creating, logging, or publishing Dataverse metadata.	It contributes to the controlled creation of the Pending Commercial Action table or lookup fix.	If changed, validate compile output and Power Apps metadata carefully.
182	}	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
183	};	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
184	(blank)	Blank readability line.	No runtime behavior; separates logical code blocks.	Improves maintainability and visual scanning.	None.
185	var lookup = new LookupAttributeMetadata	Starts lookup column metadata definition.	The lookup column stores the Opportunity reference on the child table.	Creates the visible Opportunity column in Pending Commercial Action.	Missing lookup means no direct relationship to Opportunity.
186	{	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
187	SchemaName = "aso_opportunityid",	Sets the lookup column schema name.	The column created on Pending Commercial Action is aso_opportunityid.	This was the missing column after the first script failed.	Wrong name breaks downstream scripts/flows expecting aso_opportunityid.
188	DisplayName = Label("Opportunity"),	Executes part of the script's metadata creation logic.	This line participates in defining, checking, creating, logging, or publishing Dataverse metadata.	It contributes to the controlled creation of the Pending Commercial Action table or lookup fix.	If changed, validate compile output and Power Apps metadata carefully.
189	Description = Label("Related Opportunity for the pending commercial action."),	Adds a metadata description.	Dataverse stores this explanation on the table or column metadata.	Helps future IT teams understand intent.	Descriptions are easy to skip but important for maintainability.
190	RequiredLevel = Optional()	Executes part of the script's metadata creation logic.	This line participates in defining, checking, creating, logging, or publishing Dataverse metadata.	It contributes to the controlled creation of the Pending Commercial Action table or lookup fix.	If changed, validate compile output and Power Apps metadata carefully.
191	};	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
192	(blank)	Blank readability line.	No runtime behavior; separates logical code blocks.	Improves maintainability and visual scanning.	None.
193	var request = new CreateOneToManyRequest	Creates the relationship creation request.	Combines the relationship metadata and lookup attribute into one SDK operation.	This is how Dataverse creates lookup relationships programmatically.	Incorrect metadata can create wrong relationship direction or fail validation.
194	{	Structural C# syntax.	Groups code into blocks or closes object	Required for valid C# compilation and scope	Missing braces or semicolons cause build

			initializers/statements.	control.	errors.
195	OneToManyRelationship = relationship,	Adds the relationship metadata to the request.	Passes schema, parent/child tables, menu configuration, and cascade behavior.	Required to create the relationship.	Without it, Dataverse does not know relationship structure.
196	Lookup = lookup,	Adds lookup column metadata to the request.	Passes schema, display label, description, and required level.	Creates the actual Opportunity lookup field on the child table.	Without it, no lookup column appears.
197	SolutionUniqueName = SolutionUniqueName	Associates the created component with ASO.Core.	The SDK passes the solution unique name to the create request.	Critical for export, ALM, and keeping changes out of accidental default-only context.	Wrong solution unique name means components may not appear in ASO.Core.
198	};	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
199	(blank)	Blank readability line.	No runtime behavior; separates logical code blocks.	Improves maintainability and visual scanning.	None.
200	service.Execute(request);	Executes a create request.	Sends CreateAttributeRequest or CreateOneToManyRequest to Dataverse.	Actually creates a column or relationship.	Failures here indicate permissions, invalid metadata, or language/choice issues.
201	(blank)	Blank readability line.	No runtime behavior; separates logical code blocks.	Improves maintainability and visual scanning.	None.
202	Console.ForegroundColor = ConsoleColor.Green;	Changes terminal output to green.	Used for successful creation/publishing messages.	Gives visual confirmation of completed metadata actions.	Purely cosmetic, but useful during long script runs.
203	Console.WriteLine("CREATED LOOKUP: aso_opportunityid");	Writes a status message to the terminal.	Provides an execution log for the operator.	Makes the run auditable and easier to troubleshoot.	If messages are vague, failures become harder to interpret.
204	Console.ResetColor();	Executes part of the script's metadata creation logic.	This line participates in defining, checking, creating, logging, or publishing Dataverse metadata.	It contributes to the controlled creation of the Pending Commercial Action table or lookup fix.	If changed, validate compile output and Power Apps metadata carefully.
205	}	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
206	catch (Exception ex)	Starts exception handling.	Captures SDK/Dataverse errors for the current operation.	Keeps the operator informed about exact failures.	Ignoring exceptions makes recovery difficult.
207	{	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
208	Console.ForegroundColor = ConsoleColor.Red;	Changes terminal output to red.	Used for failure messages so operators can identify errors immediately.	Improves operational readability.	If colors are not reset, later messages can remain incorrectly colored.
209	Console.WriteLine("FAILED LOOKUP: aso_opportunityid");	Writes a status message to the terminal.	Provides an execution log for the operator.	Makes the run auditable and easier to troubleshoot.	If messages are vague, failures become harder to interpret.
210	Console.WriteLine(ex.Message);	Writes a status message to the terminal.	Provides an execution log for the operator.	Makes the run auditable and easier to troubleshoot.	If messages are vague, failures become harder to interpret.
211	Console.ResetColor();	Executes part of the script's metadata creation logic.	This line participates in defining, checking, creating, logging, or publishing Dataverse metadata.	It contributes to the controlled creation of the Pending Commercial Action table or lookup fix.	If changed, validate compile output and Power Apps metadata carefully.
212	}	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
213	}	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
214	(blank)	Blank readability line.	No runtime behavior; separates logical code blocks.	Improves maintainability and visual scanning.	None.
215	static bool EntityExists(ServiceClient service, string entityLogicalName)	Defines a helper to check if a table exists.	It attempts to retrieve table metadata and returns true/false.	Prevents duplicate table creation.	Broad catch hides detailed existence-check errors; acceptable for this helper but should be used carefully.
216	{	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
217	try	Starts a protected operation block.	Exceptions inside the block are caught and logged.	Prevents one failing field from hiding the exact cause.	Without try/catch, the script may crash with less context.
218	{	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
219	service.Execute(new RetrieveEntityRequest	Creates a metadata retrieval request.	Dataverse uses it to retrieve table or attribute metadata.	Needed to check existing tables/columns.	Wrong filters may return incomplete metadata.
220	{	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
221	LogicalName = entityLogicalName,	Executes part of the script's metadata creation logic.	This line participates in defining, checking, creating, logging, or publishing Dataverse metadata.	It contributes to the controlled creation of the Pending Commercial Action table or lookup fix.	If changed, validate compile output and Power Apps metadata carefully.
222	EntityFilters = EntityFilters.Entity,	Executes part of the script's metadata creation logic.	This line participates in defining, checking, creating, logging, or publishing Dataverse metadata.	It contributes to the controlled creation of the Pending Commercial Action table or lookup fix.	If changed, validate compile output and Power Apps metadata carefully.
223	RetrieveAsIfPublished = true	Reads metadata as if all customizations are published.	Helps include recent metadata changes.	Improves reliability after script-created components.	If false, recently created unpublished metadata might not be detected.
224	});	Executes part of the script's metadata creation logic.	This line participates in defining, checking, creating, logging, or publishing Dataverse metadata.	It contributes to the controlled creation of the Pending Commercial Action table or lookup fix.	If changed, validate compile output and Power Apps metadata carefully.
225	(blank)	Blank readability line.	No runtime behavior; separates logical code blocks.	Improves maintainability and visual scanning.	None.
226	return true;	Executes part of the script's metadata creation logic.	This line participates in defining, checking, creating, logging, or publishing Dataverse	It contributes to the controlled creation of the Pending Commercial Action table or lookup	If changed, validate compile output and Power Apps metadata carefully.

			metadata.	fix.	
227	}	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
228	catch	Starts exception handling.	Captures SDK/Dataverse errors for the current operation.	Keeps the operator informed about exact failures.	Ignoring exceptions makes recovery difficult.
229	{	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
230	return false;	Executes part of the script's metadata creation logic.	This line participates in defining, checking, creating, logging, or publishing Dataverse metadata.	It contributes to the controlled creation of the Pending Commercial Action table or lookup fix.	If changed, validate compile output and Power Apps metadata carefully.
231	}	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
232	}	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
233	(blank)	Blank readability line.	No runtime behavior; separates logical code blocks.	Improves maintainability and visual scanning.	None.
234	static HashSet<string> GetExistingAttributeLogicalNames(ServiceClient service, string entityLogicalName)	Defines a helper to retrieve existing column names.	Returns a set for fast case-insensitive lookup.	Supports safe reruns and partial recovery.	If not reliable, duplicate detection fails.
235	{	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
236	var response = (RetrieveEntityResponse)service.Execute(new RetrieveEntityRequest	Creates a metadata retrieval request.	Dataverse uses it to retrieve table or attribute metadata.	Needed to check existing tables/columns.	Wrong filters may return incomplete metadata.
237	{	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
238	LogicalName = entityLogicalName,	Executes part of the script's metadata creation logic.	This line participates in defining, checking, creating, logging, or publishing Dataverse metadata.	It contributes to the controlled creation of the Pending Commercial Action table or lookup fix.	If changed, validate compile output and Power Apps metadata carefully.
239	EntityFilters = EntityFilters.Attributes,	Requests column/attribute metadata.	The response includes existing table attributes.	Needed for existing column checks.	Using only Entity filters would not return columns.
240	RetrieveAsIfPublished = true	Reads metadata as if all customizations are published.	Helps include recent metadata changes.	Improves reliability after script-created components.	If false, recently created unpublished metadata might not be detected.
241	});	Executes part of the script's metadata creation logic.	This line participates in defining, checking, creating, logging, or publishing Dataverse metadata.	It contributes to the controlled creation of the Pending Commercial Action table or lookup fix.	If changed, validate compile output and Power Apps metadata carefully.
242	(blank)	Blank readability line.	No runtime behavior; separates logical code blocks.	Improves maintainability and visual scanning.	None.
243	return response.EntityMetadata.Attributes	Starts building a list of existing attribute logical names.	Reads the metadata response returned by Dataverse.	Forms the basis of idempotency checks.	If the response is null, the helper would fail.
244	.Where(a => !string.IsNullOrWhiteSpace(a.LogicalName))	Executes part of the script's metadata creation logic.	This line participates in defining, checking, creating, logging, or publishing Dataverse metadata.	It contributes to the controlled creation of the Pending Commercial Action table or lookup fix.	If changed, validate compile output and Power Apps metadata carefully.
245	.Select(a => a.LogicalName!)	Executes part of the script's metadata creation logic.	This line participates in defining, checking, creating, logging, or publishing Dataverse metadata.	It contributes to the controlled creation of the Pending Commercial Action table or lookup fix.	If changed, validate compile output and Power Apps metadata carefully.
246	.ToHashSet(StringComparer.OrdinalIgnoreCase);	Executes part of the script's metadata creation logic.	This line participates in defining, checking, creating, logging, or publishing Dataverse metadata.	It contributes to the controlled creation of the Pending Commercial Action table or lookup fix.	If changed, validate compile output and Power Apps metadata carefully.
247	}	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
248	(blank)	Blank readability line.	No runtime behavior; separates logical code blocks.	Improves maintainability and visual scanning.	None.
249	static Label Label(string value) => new(value, LanguageCode);	Defines a helper for localized labels.	Creates Dataverse Label objects using the configured LanguageCode.	Avoids repeating label creation logic everywhere.	Using unsupported LanguageCode caused the initial lookup failure.
250	(blank)	Blank readability line.	No runtime behavior; separates logical code blocks.	Improves maintainability and visual scanning.	None.
251	static AttributeRequiredLevelManagedProperty Optional()	Defines a helper for optional fields.	Returns AttributeRequiredLevel.None wrapped in a Dataverse managed property.	Keeps most custom columns optional for MVP flexibility.	Making fields required too early can break integrations and tests.
252	{	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
253	return new AttributeRequiredLevelManagedProperty(AttributeRequiredLevel.None);	Executes part of the script's metadata creation logic.	This line participates in defining, checking, creating, logging, or publishing Dataverse metadata.	It contributes to the controlled creation of the Pending Commercial Action table or lookup fix.	If changed, validate compile output and Power Apps metadata carefully.
254	}	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
255	(blank)	Blank readability line.	No runtime behavior; separates logical code blocks.	Improves maintainability and visual scanning.	None.
256	static StringAttributeMetadata Text(string schemaName, string displayName, int maxLength, string description)	Defines the helper for single-line text fields.	Returns StringAttributeMetadata with schema, labels, description, requirement level, and max length.	Avoids repetitive code for text columns.	Incorrect max length or schema affects data quality.

257	{	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
258	return new StringAttributeMetadata	Executes part of the script's metadata creation logic.	This line participates in defining, checking, creating, logging, or publishing Dataverse metadata.	It contributes to the controlled creation of the Pending Commercial Action table or lookup fix.	If changed, validate compile output and Power Apps metadata carefully.
259	{	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
260	SchemaName = schemaName,	Executes part of the script's metadata creation logic.	This line participates in defining, checking, creating, logging, or publishing Dataverse metadata.	It contributes to the controlled creation of the Pending Commercial Action table or lookup fix.	If changed, validate compile output and Power Apps metadata carefully.
261	DisplayName = Label(displayName),	Executes part of the script's metadata creation logic.	This line participates in defining, checking, creating, logging, or publishing Dataverse metadata.	It contributes to the controlled creation of the Pending Commercial Action table or lookup fix.	If changed, validate compile output and Power Apps metadata carefully.
262	Description = Label(description),	Adds a metadata description.	Dataverse stores this explanation on the table or column metadata.	Helps future IT teams understand intent.	Descriptions are easy to skip but important for maintainability.
263	RequiredLevel = Optional(),	Executes part of the script's metadata creation logic.	This line participates in defining, checking, creating, logging, or publishing Dataverse metadata.	It contributes to the controlled creation of the Pending Commercial Action table or lookup fix.	If changed, validate compile output and Power Apps metadata carefully.
264	MaxLength = maxLength	Sets the maximum text length.	Dataverse validates text length at metadata and runtime levels.	Prevents uncontrolled field sizes and follows the data model.	Too small truncates useful data; too large may reduce data quality.
265	};	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
266	}	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
267	(blank)	Blank readability line.	No runtime behavior; separates logical code blocks.	Improves maintainability and visual scanning.	None.
268	static MemoAttributeMetadata Memo(string schemaName, string displayName, string description)	Defines the helper for multiline text fields.	Returns MemoAttributeMetadata with schema, labels, description, requirement level, and length.	Used for payloads and error messages.	Short text would not be sufficient for these values.
269	{	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
270	return new MemoAttributeMetadata	Executes part of the script's metadata creation logic.	This line participates in defining, checking, creating, logging, or publishing Dataverse metadata.	It contributes to the controlled creation of the Pending Commercial Action table or lookup fix.	If changed, validate compile output and Power Apps metadata carefully.
271	{	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
272	SchemaName = schemaName,	Executes part of the script's metadata creation logic.	This line participates in defining, checking, creating, logging, or publishing Dataverse metadata.	It contributes to the controlled creation of the Pending Commercial Action table or lookup fix.	If changed, validate compile output and Power Apps metadata carefully.
273	DisplayName = Label(displayName),	Executes part of the script's metadata creation logic.	This line participates in defining, checking, creating, logging, or publishing Dataverse metadata.	It contributes to the controlled creation of the Pending Commercial Action table or lookup fix.	If changed, validate compile output and Power Apps metadata carefully.
274	Description = Label(description),	Adds a metadata description.	Dataverse stores this explanation on the table or column metadata.	Helps future IT teams understand intent.	Descriptions are easy to skip but important for maintainability.
275	RequiredLevel = Optional(),	Executes part of the script's metadata creation logic.	This line participates in defining, checking, creating, logging, or publishing Dataverse metadata.	It contributes to the controlled creation of the Pending Commercial Action table or lookup fix.	If changed, validate compile output and Power Apps metadata carefully.
276	MaxLength = 4000	Sets the maximum text length.	Dataverse validates text length at metadata and runtime levels.	Prevents uncontrolled field sizes and follows the data model.	Too small truncates useful data; too large may reduce data quality.
277	};	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
278	}	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
279	(blank)	Blank readability line.	No runtime behavior; separates logical code blocks.	Improves maintainability and visual scanning.	None.
280	static DateTimeAttributeMetadata DateTimeField(string schemaName, string displayName, string description)	Defines the helper for date/time fields.	Returns DateTimeAttributeMetadata with DateAndTime format and UserLocal behavior.	Used for Submitted On and other timestamps.	Wrong behavior can confuse cross-time-zone users.
281	{	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
282	return new DateTimeAttributeMetadata	Executes part of the script's metadata creation logic.	This line participates in defining, checking, creating, logging, or publishing Dataverse metadata.	It contributes to the controlled creation of the Pending Commercial Action table or lookup fix.	If changed, validate compile output and Power Apps metadata carefully.
283	{	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
284	SchemaName = schemaName,	Executes part of the script's metadata creation logic.	This line participates in defining, checking, creating, logging, or publishing Dataverse metadata.	It contributes to the controlled creation of the Pending Commercial Action table or lookup fix.	If changed, validate compile output and Power Apps metadata carefully.
285	DisplayName = Label(displayName),	Executes part of the script's metadata creation logic.	This line participates in defining, checking, creating, logging, or publishing Dataverse metadata.	It contributes to the controlled creation of the Pending Commercial Action table or lookup fix.	If changed, validate compile output and Power Apps metadata carefully.
286	Description = Label(description),	Adds a metadata description.	Dataverse stores this explanation on the	Helps future IT teams understand intent.	Descriptions are easy to skip but important

			table or column metadata.		for maintainability.
287	RequiredLevel = Optional(),	Executes part of the script's metadata creation logic.	This line participates in defining, checking, creating, logging, or publishing Dataverse metadata.	It contributes to the controlled creation of the Pending Commercial Action table or lookup fix.	If changed, validate compile output and Power Apps metadata carefully.
288	Format = DateTimeFormat.DateAndTime,	Executes part of the script's metadata creation logic.	This line participates in defining, checking, creating, logging, or publishing Dataverse metadata.	It contributes to the controlled creation of the Pending Commercial Action table or lookup fix.	If changed, validate compile output and Power Apps metadata carefully.
289	DateTimeBehavior = DateTimeBehavior.UserLocal	Executes part of the script's metadata creation logic.	This line participates in defining, checking, creating, logging, or publishing Dataverse metadata.	It contributes to the controlled creation of the Pending Commercial Action table or lookup fix.	If changed, validate compile output and Power Apps metadata carefully.
290	};	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
291	}	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
292	(blank)	Blank readability line.	No runtime behavior; separates logical code blocks.	Improves maintainability and visual scanning.	None.
293	static PicklistAttributeMetadata LocalChoice(string schemaName, string displayName, string[] values, string description)	Defines the helper for local choice columns.	Creates a PicklistAttributeMetadata object backed by a non-global OptionSetMetadata.	Used for table-specific action/status choices.	If a reusable taxonomy is needed later, a global choice may be preferable.
294	{	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
295	var optionSet = new OptionSetMetadata	Executes part of the script's metadata creation logic.	This line participates in defining, checking, creating, logging, or publishing Dataverse metadata.	It contributes to the controlled creation of the Pending Commercial Action table or lookup fix.	If changed, validate compile output and Power Apps metadata carefully.
296	{	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
297	IsGlobal = false,	Marks the option set as local to the column.	The choice values live on this field only.	Appropriate for Pending Commercial Action-specific choices.	Do not use local choices when cross-table consistency is required.
298	OptionSetType = OptionSetType.Picklist	Specifies a standard choice/picklist option set.	Dataverse supports multiple option set types; this one is for single-choice fields.	Needed for Action Type and Status fields.	Wrong option set type causes invalid metadata.
299	};	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
300	(blank)	Blank readability line.	No runtime behavior; separates logical code blocks.	Improves maintainability and visual scanning.	None.
301	foreach (var value in values)	Executes part of the script's metadata creation logic.	This line participates in defining, checking, creating, logging, or publishing Dataverse metadata.	It contributes to the controlled creation of the Pending Commercial Action table or lookup fix.	If changed, validate compile output and Power Apps metadata carefully.
302	{	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
303	optionSet.Options.Add(new OptionMetadata(Label(value), null));	Adds one option value to the choice field.	Each string in the values array becomes an OptionMetadata entry.	Creates the selectable values visible in Power Apps.	Incorrect labels create wrong user-facing status/action values.
304	}	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
305	(blank)	Blank readability line.	No runtime behavior; separates logical code blocks.	Improves maintainability and visual scanning.	None.
306	return new PicklistAttributeMetadata	Executes part of the script's metadata creation logic.	This line participates in defining, checking, creating, logging, or publishing Dataverse metadata.	It contributes to the controlled creation of the Pending Commercial Action table or lookup fix.	If changed, validate compile output and Power Apps metadata carefully.
307	{	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
308	SchemaName = schemaName,	Executes part of the script's metadata creation logic.	This line participates in defining, checking, creating, logging, or publishing Dataverse metadata.	It contributes to the controlled creation of the Pending Commercial Action table or lookup fix.	If changed, validate compile output and Power Apps metadata carefully.
309	DisplayName = Label(displayName),	Executes part of the script's metadata creation logic.	This line participates in defining, checking, creating, logging, or publishing Dataverse metadata.	It contributes to the controlled creation of the Pending Commercial Action table or lookup fix.	If changed, validate compile output and Power Apps metadata carefully.
310	Description = Label(description),	Adds a metadata description.	Dataverse stores this explanation on the table or column metadata.	Helps future IT teams understand intent.	Descriptions are easy to skip but important for maintainability.
311	RequiredLevel = Optional(),	Executes part of the script's metadata creation logic.	This line participates in defining, checking, creating, logging, or publishing Dataverse metadata.	It contributes to the controlled creation of the Pending Commercial Action table or lookup fix.	If changed, validate compile output and Power Apps metadata carefully.
312	OptionSet = optionSet	Executes part of the script's metadata creation logic.	This line participates in defining, checking, creating, logging, or publishing Dataverse metadata.	It contributes to the controlled creation of the Pending Commercial Action table or lookup fix.	If changed, validate compile output and Power Apps metadata carefully.
313	};	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
314	}	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
315	EOF	Executes part of the script's metadata creation logic.	This line participates in defining, checking, creating, logging, or publishing Dataverse metadata.	It contributes to the controlled creation of the Pending Commercial Action table or lookup fix.	If changed, validate compile output and Power Apps metadata carefully.

Appendix - Full initial script

```
using Microsoft.Crm.Sdk.Messages;
using Microsoft.PowerPlatform.Dataverse.Client;
using Microsoft.Xrm.Sdk;
using Microsoft.Xrm.Sdk.Messages;
using Microsoft.Xrm.Sdk.Metadata;

const string DataverseUrl = "https://phoenicarix-ci.crm4.dynamics.com";
const string SolutionUniqueName = "ASOCore";
const string EntityLogicalName = "aso_pendingcommercialaction";
const int LanguageCode = 1033;

var connectionString =
    $@"AuthType=OAuth;
       Url={DataverseUrl};
       LoginPrompt=Auto;
       ClientId=51f81489-12ee-4a9e-aaae-a2591f45987d;
       RedirectUri=http://localhost";

using var service = new ServiceClient(connectionString);

if (!service.IsReady)
{
    Console.ForegroundColor = ConsoleColor.Red;
    Console.WriteLine("Connection failed:");
    Console.WriteLine(service.LastError);
    Console.ResetColor();
    return;
}

Console.WriteLine($"Connected to: {DataverseUrl}");
Console.WriteLine($"Target solution: {SolutionUniqueName}");
Console.WriteLine($"Target table: {EntityLogicalName}");
Console.WriteLine();

if (!EntityExists(service, EntityLogicalName))
{
    Console.WriteLine("Creating Pending Commercial Action table...");

    var createEntityRequest = new CreateEntityRequest
    {
        Entity = new EntityMetadata
        {
            SchemaName = "aso_pendingcommercialaction",
            DisplayName = Label("Pending Commercial Action"),
            DisplayCollectionName = Label("Pending Commercial Actions"),
            Description = Label("Approval-gated commercial action staging table before SAP submit."),
            OwnershipType = OwnershipTypes.UserOwned
        },
        PrimaryAttribute = new StringAttributeMetadata
        {
            SchemaName = "aso_name",
            DisplayName = Label("Name"),
            Description = Label("Primary name for the Pending Commercial Action record."),
            RequiredLevel = new AttributeRequiredLevelManagedProperty(AttributeRequiredLevel.ApplicationRequired),
            MaxLength = 200
        },
        SolutionUniqueName = SolutionUniqueName
    };

    service.Execute(createEntityRequest);

    Console.ForegroundColor = ConsoleColor.Green;
    Console.WriteLine("CREATED TABLE: aso_pendingcommercialaction");
    Console.ResetColor();
}
else
{
    Console.ForegroundColor = ConsoleColor.Yellow;
    Console.WriteLine("SKIP existing table: aso_pendingcommercialaction");
    Console.ResetColor();
```

```
}

CreateOpportunityLookupIfMissing(service);

var existingAttributes = GetExistingAttributeLogicalNames(service, EntityLogicalName);

var fields = new List<AttributeMetadata>
{
    LocalChoice("aso_actiontype", "Action Type",
        new[] { "CreateOrder", "UpdateCommercialReference", "ReviewPricingContext" },
        "Commercial action type staged for approval before SAP submission."),

    Memo("aso_payload", "Payload", "Commercial action payload staged before approval and SAP submission."),

    LocalChoice("aso_status", "Status",
        new[] { "Draft", "AwaitingApproval", "Approved", "Submitted", "Failed", "Cancelled" },
        "Approval and submission status of the pending commercial action."),

    Text("aso_sapdocumentid", "SAP Document ID", 100, "SAP document identifier returned after governed SAP submit."),
    Memo("aso_errormessage", "Error Message", "Error details if approval or SAP submission fails."),
    Text("aso_approvalid", "Approval ID", 100, "Power Automate or approval process identifier."),
    Text("aso_idempotencykey", "Idempotency Key", 200, "Key used to prevent duplicate commercial submission."),
    DateTimeField("aso_submittedon", "Submitted On", "Timestamp when the commercial action was submitted.")
};

foreach (var field in fields)
{
    var logicalName = field.SchemaName!.ToLowerInvariant();

    if (existingAttributes.Contains(logicalName))
    {
        Console.ForegroundColor = ConsoleColor.Yellow;
        Console.WriteLine($"SKIP existing: {logicalName}");
        Console.ResetColor();
        continue;
    }

    try
    {
        Console.WriteLine($"Creating: {logicalName} ...");

        var request = new CreateAttributeRequest
        {
            EntityName = EntityLogicalName,
            Attribute = field,
            SolutionUniqueName = SolutionUniqueName
        };

        service.Execute(request);

        Console.ForegroundColor = ConsoleColor.Green;
        Console.WriteLine($"CREATED: {logicalName}");
        Console.ResetColor();
    }
    catch (Exception ex)
    {
        Console.ForegroundColor = ConsoleColor.Red;
        Console.WriteLine($"FAILED: {logicalName}");
        Console.WriteLine(ex.Message);
        Console.ResetColor();
    }
}

Console.WriteLine();
Console.WriteLine("Publishing Pending Commercial Action table customizations...");

service.Execute(new PublishXmlRequest
{
    ParameterXml = "<importexportxml><entities><entity>aso_pendingcommercialaction</entity><entity>opportunity</entity></entities></importexportxml>"
});

Console.ForegroundColor = ConsoleColor.Green;
Console.WriteLine("Done. Validate in ASO.Core -> Tables -> Pending Commercial Action -> Columns.");
Console.ResetColor();

void CreateOpportunityLookupIfMissing(ServiceClient service)
```

```
{
    var existingAttributes = GetExistingAttributeLogicalNames(service, EntityLogicalName);

    if (existingAttributes.Contains("aso_opportunityid"))
    {
        Console.ForegroundColor = ConsoleColor.Yellow;
        Console.WriteLine("SKIP existing lookup: aso_opportunityid");
        Console.ResetColor();
        return;
    }

    try
    {
        Console.WriteLine("Creating Opportunity lookup relationship...");

        var relationship = new OneToManyRelationshipMetadata
        {
            SchemaName = "aso_opportunity_aso_pendingcommercialaction",
            ReferencedEntity = "opportunity",
            ReferencingEntity = EntityLogicalName,
            AssociatedMenuConfiguration = new AssociatedMenuConfiguration
            {
                Behavior = AssociatedMenuBehavior.UseLabel,
                Group = AssociatedMenuGroup.Details,
                Label = Label("Pending Commercial Actions"),
                Order = 10000
            },
            CascadeConfiguration = new CascadeConfiguration
            {
                Assign = CascadeType.NoCascade,
                Delete = CascadeType.RemoveLink,
                Merge = CascadeType.NoCascade,
                Reparent = CascadeType.NoCascade,
                Share = CascadeType.NoCascade,
                Unshare = CascadeType.NoCascade
            }
        };

        var lookup = new LookupAttributeMetadata
        {
            SchemaName = "aso_opportunityid",
            DisplayName = Label("Opportunity"),
            Description = Label("Related Opportunity for the pending commercial action."),
            RequiredLevel = Optional()
        };

        var request = new CreateOneToManyRequest
        {
            OneToManyRelationship = relationship,
            Lookup = lookup,
            SolutionUniqueName = SolutionUniqueName
        };

        service.Execute(request);

        Console.ForegroundColor = ConsoleColor.Green;
        Console.WriteLine("CREATED LOOKUP: aso_opportunityid");
        Console.ResetColor();
    }
    catch (Exception ex)
    {
        Console.ForegroundColor = ConsoleColor.Red;
        Console.WriteLine("FAILED LOOKUP: aso_opportunityid");
        Console.WriteLine(ex.Message);
        Console.ResetColor();
    }
}

static bool EntityExists(ServiceClient service, string entityLogicalName)
{
    try
    {
        service.Execute(new RetrieveEntityRequest
        {
            LogicalName = entityLogicalName,
            EntityFilters = EntityFilters.Entity,
```

```
            RetrieveAsIfPublished = true
        });

        return true;
    }
    catch
    {
        return false;
    }
}

static HashSet<string> GetExistingAttributeLogicalNames(ServiceClient service, string entityLogicalName)
{
    var response = (RetrieveEntityResponse)service.Execute(new RetrieveEntityRequest
    {
        LogicalName = entityLogicalName,
        EntityFilters = EntityFilters.Attributes,
        RetrieveAsIfPublished = true
    });

    return response.EntityMetadata.Attributes
        .Where(a => !string.IsNullOrWhiteSpace(a.LogicalName))
        .Select(a => a.LogicalName!)
        .ToHashSet(StringComparer.OrdinalIgnoreCase);
}

static Label Label(string value) => new(value, LanguageCode);

static AttributeRequiredLevelManagedProperty Optional()
{
    return new AttributeRequiredLevelManagedProperty(AttributeRequiredLevel.None);
}

static StringAttributeMetadata Text(string schemaName, string displayName, int maxLength, string description)
{
    return new StringAttributeMetadata
    {
        SchemaName = schemaName,
        DisplayName = Label(displayName),
        Description = Label(description),
        RequiredLevel = Optional(),
        MaxLength = maxLength
    };
}

static MemoAttributeMetadata Memo(string schemaName, string displayName, string description)
{
    return new MemoAttributeMetadata
    {
        SchemaName = schemaName,
        DisplayName = Label(displayName),
        Description = Label(description),
        RequiredLevel = Optional(),
        MaxLength = 4000
    };
}

static DateTimeAttributeMetadata DateTimeField(string schemaName, string displayName, string description)
{
    return new DateTimeAttributeMetadata
    {
        SchemaName = schemaName,
        DisplayName = Label(displayName),
        Description = Label(description),
        RequiredLevel = Optional(),
        Format = DateTimeFormat.DateAndTime,
        DateTimeBehavior = DateTimeBehavior.UserLocal
    };
}

static PicklistAttributeMetadata LocalChoice(string schemaName, string displayName, string[] values, string description)
{
    var optionSet = new OptionSetMetadata
    {
        IsGlobal = false,
        OptionSetType = OptionSetType.Picklist
```

```
    };

    foreach (var value in values)
    {
        optionSet.Options.Add(new OptionMetadata(Label(value), null));
    }

    return new PicklistAttributeMetadata
    {
        SchemaName = schemaName,
        DisplayName = Label(displayName),
        Description = Label(description),
        RequiredLevel = Optional(),
        OptionSet = optionSet
    };
}
EOF
```